# Project 2

EE4308: Autonomous Robot Systems

**ID: A0299063U**
**ID: A0298841M**
**ID: A0296420E**


April 7, 2025

# Contents

# Abstract

This project implements a simulation of a drone equipped with a simplified Kalman filter-based state estimator within a ROS2 environment. The objective is to design a drone behavior system capable of autonomous waypoint navigation in coordination with a moving ground robot (turtlebot), while relying solely on onboard sensor data, such as IMU, sonar, GPS, barometer, and magnetometer. The overall architecture comprises three primary nodes: a behavior planner, a controller for trajectory tracking, and an estimator that fuses multiple sensor inputs to produce reliable pose estimates. The drone must take off to a designated cruise height, repeatedly visit waypoints relative to the turtlebot, and ultimately return and land once the turtlebot halts. Through systematic design of the behavior logic, controller algorithms, and estimation processes, the project demonstrates a functional multi-agent system with realistic sensor fusion. Experimental results validate the effectiveness of the simplified Kalman filter and the robustness of the integrated autonomy pipeline.
State estimation, Kalman filter, Trajectory tracking, Autonomous control, Behavior planning



*Keywords* :State Estimation, Kalman Filter, Trajectory Tracking, Autonomous Control, Behavior Planning

# 1 Introduction

This project, conducted as part of the EE4308 Autonomous Robot Systems course, aims to simulate a drone navigating in a three-dimensional space with limited perception, while interacting with a mobile ground robot (turtlebot). The core of the project lies in the use of a simplified Kalman filter for state estimation, which fuses data from inertial and absolute sensors to generate accurate position and velocity estimates.

The system is decomposed into three main ROS2 nodes:

1. Behavior Node – Implements a finite state machine for task sequencing, including take-off, following the turtlebot, and landing.
2. Controller Node – Responsible for following paths using velocity commands to track planned trajectories.
3. Estimator Node – Implements the prediction-correction cycle of the simplified Kalman filter, handling various sensor callbacks to update the drone's belief of its state.

The mission objective is to enable the drone to autonomously track and navigate to a moving turtlebot and its destination waypoint in cycles, and finally perform a safe landing after the turtlebot completes its mission.

The rest of the report is structured as follows: Section 2 details the behavior logic and state transitions. Section 3 explains the controller design used for trajectory tracking. Section 4 presents the implementation of the Kalman filter-based state estimator, which also include the sensor fusion strategy that integrates measurements from multiple onboard sensors such as IMU and GPS.

# 2 Behavior Node

In autonomous robotics, coordinated behaviors between aerial and ground robots can significantly enhance mission capabilities. In this part, we focus on the aerial component, where a drone monitors and tracks a TurtleBot by subscribing to its path and reacting to stop signals. The drone's behavior is modeled as a finite state machine and implemented in the Behavior class.

## 2.1 Finite State Machine Design

The drone operates within a ROS2 framework, where it interacts with the environment through published and subscribed topics, and communicates with a planning service to obtain navigation paths. The behavior of the drone is governed by a finite state machine (FSM) with the following states:

1. BEGIN: Initial setup, waiting for odometry data.
2. TAKEOFF: Ascends to a predefined cruise height at the initial (x, y) location.
3. INITIAL: Moves to the front of the TurtleBot's path.
4. TURTLE_POSITION: Moves towards the TurtleBot's current position.
5. TURTLE_WAYPOINT: Tracks the TurtleBot along its planned path.
6. LANDING: Descends vertically to the original z-height.
7. END: Terminates operations and shuts down the timer.

State transitions are triggered based on the drone's proximity to the target waypoint or upon receiving a stop signal from the TurtleBot.

## 2.2 Behavioral Logic

In the cbTimer callback, the drone calculates the Euclidean distance to its current waypoint. When the distance falls below a specified threshold (reached_thres_), a state transition is triggered. The transitions are as follows:

1. TAKEOFF → INITIAL
2. INITIAL → TURTLE_POSITION
3. TURTLE_POSITION → TURTLE_WAYPOINT
4. TURTLE_WAYPOINT → TURTLE_POSITION (loop), or → LANDING (if TurtleBot stops)
5. LANDING → END

The above part is the core logic of the Drone mission, that is, scheduling and controlling the mission state through the Finite State Machine, FSM. This part (we call it FSM_update function) is called periodically in the timer callback to perform corresponding actions according to the current state and the received message, and dynamically switch the state according to the task progress. The pseudo code of the FSM_update function corresponds to the following:

**Algorithm 1** FSM_update()

```
1:  procedure FSM_UPDATE
2:      switch current_state
3:        case INIT:
4:      if reached(start_point) then
5:          transition_to(TRACK)
6:      else
7:          go_to(start_point)
8:      end if
9:        case TRACK:
10:     if received(turtlebot_path) then
11:         replan_path(turtlebot_path)
12:     end if
13:     follow_current_path()
14:     if received(stop_signal) then
15:         transition_to(LAND)
16:     end if
17:       case LAND:
18:     if not is_landing then
19:         start_landing()
20:     end if
21:     if landed_successfully then
22:         transition_to(FINISH)
23:     end if
24:       case FINISH:
25:     stop_timer()
26:     disarm_uav()
27:     end switch
28: end procedure
```

### 2.3 Initialization and Execution Flow

The following pseudo code describes the complete behavior logic of the drone from start to completion of the mission, which ensure that the drone can adapt to the dynamic environment and respond to the task changes in real time.

**Algorithm 2** Drone Behavior Control Flow: UAV_Main_Loop()

```
1: procedure UAV_MAIN_LOOP
2:     wait for current_pose                          ▷ Get the current position
3:     wait for start_point and goal      ▷ Get the initial position and target point
4:     arm_uav()
5:     takeoff to pre-defined height
6:     initialize timer(callback_interval = k seconds)
7:     subscribe to turtlebot_path_topic
8:     subscribe to stop_signal_topic
9:     while rclcpp::ok() do
10:        FSM_update()                               ▷ The transition Function
11:    end while
12: end procedure
```

## 3 Controller Node

To implement the Controller Node, there are two main functions of the pure pursuit. The first one is finding the lookahead points and the other one is determining the movement of the drone. The basic pseudocode can be seen in 3.

**Algorithm 3** Controller::cbTimer

```
1: function CONTROLLER::CBTIMER()
2:     Input: odom_.pose, plan_.poses
3:     if controller is disabled then
4:         Return
5:     else if path is empty then
6:         Stop the drone in the air.
7:         Return
8:     end if
9:     Find the closest point along the path.
10:    From the lookahead point by searching from the closest point.
11:    Determine the x and y velocities in the drone's frame to reach the lookahead point.
12:    Determine the z velocity in the drone's frame to reach the lookahead point.
13:    Constrain the x and y velocities.
14:    Constrain the z velocity.
15:    Move the drone in x, y, and z, and at the required yaw velocity.
16: end function
```

### 3.1 Lookahead Points

Unlike in Project 1, the front of the path (index 0) corresponds to the drone's current position, while the back of the path represents the desired waypoint. The path is only updated when the waypoint changes. Therefore, unless the waypoint is dynamically attached to a moving target (e.g., the turtle), the path remains static most of the time.

As a result, the lookahead point should be determined by first locating the closest point on the path to the drone. The procedure is as follows:

1 Identify the point on the path that is closest to the drone's current position.

2 From this closest point, traverse the path toward the desired waypoint (i.e., the back of the path) and find the first point whose distance from the drone exceeds the predefined lookahead distance. This point becomes the lookahead point.

3 If no such point is found, it implies that the drone is near the desired waypoint. In this case, the lookahead point is set to the waypoint itself.

### 3.2 Movement of Drone

Since the drone is holonomic and can move in any direction, the curvature from project 1 should not be calculated. The desired velocities are first determined by a proportional$P$ controller, which considers the distance to the lookahead point. Then the desired velocities are subsequently constrained. There is a key point to transform the coordinate frame from the world frame to the drone body frame.

$$dx_{body} = \cos(\psi) \cdot dx_{world} + \sin(\psi) \cdot dy_{world} \tag{1a}$$

$$dy_{body} = -\sin(\psi) \cdot dx_{world} + \cos(\psi) \cdot dy_{world} \tag{1b}$$

## 4 Estimator Node

Accurate state estimation is essential for autonomous navigation. In this part, we implement an Extended Kalman Filter (EKF) to estimate the drone's 3D position, velocity, and yaw angle based on sensor data from IMU and GPS. The implementation is encapsulated in the Estimator class and operates within a ROS2 framework.

### 4.1 Prediction Step using IMU

The `cbIMU()` callback is responsible for the prediction step of the EKF. It receives linear acceleration and angular velocity in the drone's body frame. The algorithm transforms the acceleration into the world frame using the current yaw angle, then integrates it to update the velocity and position for each of the $x$, $y$, and $z$ axes. Yaw angle is updated using the angular velocity around the $z$-axis.

The prediction step of the EKF uses the following state transition equations:

$$\mathbf{X}_k = \mathbf{F} \cdot \mathbf{X}_{k-1} + \mathbf{B} \cdot \mathbf{u}_k + \mathbf{w}_k \tag{2}$$

$$\mathbf{P}_k = \mathbf{F} \cdot \mathbf{P}_{k-1} \cdot \mathbf{F}^\top + \mathbf{Q} \tag{3}$$

Where $\mathbf{F} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}$ is the state transition matrix, $\mathbf{u} = a$ is the acceleration input, and $\mathbf{B} = \begin{bmatrix} \frac{1}{2}\Delta t^2 \\ \Delta t \end{bmatrix}$ is the input matrix. Yaw angle $\theta$ is updated as:

$$\theta_k = \theta_{k-1} + \omega_z \cdot \Delta t, \quad \theta_k \in [-\pi, \pi] \tag{4}$$

### 4.2 GPS Correction and ECEF Conversion

The `cbGPS()` callback handles the correction step using global GPS measurements. The raw latitude, longitude, and altitude are converted to Earth-Centered Earth-Fixed (ECEF) coordinates using the WGS-84 ellipsoid model in the `getECEF()` function. These coordinates are then projected into the local ENU (East-North-Up) frame using a rotation matrix.

The GPS update process includes the following coordinate transformations:

**Algorithm 4** EKF_Predict(IMU)

**procedure** EKF_Predict(IMU_msg)
  $dt \leftarrow$ time since last prediction
  $\theta \leftarrow$ current yaw
  $a_x \leftarrow a_{bx} \cdot \cos\theta - a_{by} \cdot \sin\theta$
  $a_y \leftarrow a_{bx} \cdot \sin\theta + a_{by} \cdot \cos\theta$
  $v_z \leftarrow v_z + (a_{bz} + g) \cdot dt$
  update $v_x, v_y$ using $a_x, a_y$
  update $x, y, z$ using $v_x, v_y, v_z$
  $\theta \leftarrow \theta + \omega_z \cdot dt$, then wrap to $[-\pi, \pi]$
  propagate $P_x, P_y, P_z, P_\theta$ with $F$ and $Q$
**end procedure**

**ECEF to ENU**:

$$\mathbf{R}_{e/n} = \begin{bmatrix} -\sin(\varphi)\cos(\lambda) & -\sin(\varphi)\sin(\lambda) & \cos(\varphi) \\ -\sin(\lambda) & \cos(\lambda) & 0 \\ -\cos(\varphi)\cos(\lambda) & -\cos(\varphi)\sin(\lambda) & -\sin(\varphi) \end{bmatrix} \tag{5}$$

$$\begin{bmatrix} x_n \\ y_n \\ z_n \end{bmatrix} = \mathbf{R}_{e/n} \cdot \left( \begin{bmatrix} x_e \\ y_e \\ z_e \end{bmatrix} - \begin{bmatrix} x_{e0} \\ y_{e0} \\ z_{e0} \end{bmatrix} \right) \tag{6}$$

**ENU to World**:

$$\mathbf{R}_{m/n} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & -1 \end{bmatrix} \tag{7}$$

$$\begin{bmatrix} x_{gps} \\ y_{gps} \\ z_{gps} \end{bmatrix} = \mathbf{R}_{m/n} \cdot \begin{bmatrix} x_n \\ y_n \\ z_n \end{bmatrix} + \begin{bmatrix} x_0 \\ y_0 \\ z_0 \end{bmatrix} \tag{8}$$

**Algorithm 5** EKF_Correct(GPS)

**procedure** EKF_Correct(GPS_msg)
  convert $(\phi, \lambda, h)$ to ECEF: $X_{ecef}$
  **if** first GPS **then**
    store $X_{ecef}$ as reference and return
  **end if**
  $ENU \leftarrow R \cdot (X_{ecef} - X_{ref}) + initial\_position$
  $Y_{gps} \leftarrow ENU$
  **for** $axis \in \{x, y, z\}$ **do**
    $K \leftarrow PH^T/(HPH^T + R)$
    $X \leftarrow X + K(Y - HX)$
    $P \leftarrow (I - KH)P$
  **end for**
**end procedure**

## 4.3 getECEF(): Coordinate Conversion

The `getECEF()` function computes ECEF coordinates from given latitude, longitude, and altitude using the ellipsoidal Earth model. It uses the following equations:

$$e^2 = 1 - \frac{b^2}{a^2} \tag{9}$$

$$N = \frac{a}{\sqrt{1 - e^2 \sin^2(\phi)}} \tag{10}$$

$$X = (N + h)\cos(\phi)\cos(\lambda) \tag{11a}$$
$$Y = (N + h)\cos(\phi)\sin(\lambda) \tag{11b}$$
$$Z = \left(N(1 - e^2) + h\right)\sin(\phi) \tag{11c}$$

where $a$ is the equatorial radius and $e$ is the eccentricity of Earth.

**Algorithm 6** getECEF(lat, lon, alt)

**procedure** GETECEF(lat, lon, alt)
    Convert lat, lon to radians
    $N \leftarrow \frac{a}{\sqrt{1-e^2 \sin^2(lat)}}$
    $x \leftarrow (N + alt) \cdot \cos(lat) \cdot \cos(lon)$
    $y \leftarrow (N + alt) \cdot \cos(lat) \cdot \sin(lon)$
    $z \leftarrow \left(N(1 - e^2) + alt\right) \cdot \sin(lat)$
    **return** $(x, y, z)$
**end procedure**

## 4.4 Detailed EKF Prediction for Each Axis

The EKF prediction step applies the same structure to $x$, $y$, $z$, and yaw $\theta$ states, but is based on IMU input specific to each direction.

For example, for $x$-axis:

$$\hat{x}_k = x_{k-1} + v_{x,k-1} \cdot \Delta t + \frac{1}{2} a_x \cdot \Delta t^2 \tag{12}$$

$$v_{x,k} = v_{x,k-1} + a_x \cdot \Delta t \tag{13}$$

with corresponding state covariance update:

$$F = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}, \quad Q = \sigma_{a_x}^2 \cdot \begin{bmatrix} \frac{1}{4}\Delta t^4 & \frac{1}{2}\Delta t^3 \\ \frac{1}{2}\Delta t^3 & \Delta t^2 \end{bmatrix} \tag{14}$$

The same logic applies independently for $y$ and $z$ axes using $a_y$, $a_z$, and corresponding noise variances $\sigma_{a_y}^2$, $\sigma_{a_z}^2$.

For yaw update:

$$\theta_k = \theta_{k-1} + \omega_z \cdot \Delta t, \quad \theta_k \in [-\pi, \pi] \tag{15}$$

## 4.5 Details in GPS ECEF to ENU Projection

After ECEF conversion from GPS, the vector from the reference point is:

$$\Delta \mathbf{X}_{ecef} = \mathbf{X}_{ecef} - \mathbf{X}_{ecef,0} \tag{16}$$

Then the ENU position is obtained by:

$$\mathbf{X}_{enu} = R_{e/n} \cdot \Delta \mathbf{X}_{ecef} \tag{17}$$

Where $R_{e/n}$ is the ECEF to ENU rotation matrix parameterized by the initial latitude and longitude.

Finally, we convert ENU to simulation-aligned world frame using:

$$\mathbf{X}_{world} = R_{m/n} \cdot \mathbf{X}_{enu} + \mathbf{X}_{init} \tag{18}$$

### 4.6 Height Correction Using Sonar

The sonar sensor can be used to measure the drone's height from the ground. By emitting sound waves and receiving the echoes from the ground, the sensor calculates the distance to the surface, providing accurate altitude measurements. This is essential for stable hovering and terrain-following, especially in situations where GPS signals are weak or unavailable. In our project, the sonar sensor works alongside other sensors, such as GPS and IMU, to assist with precise positioning.

The Sonar sensor model can be seen in equations 4.6. The goal is to update the state of the system based on noisy sensor measurements. The state is predicted in advance, and the sensor provides a noisy measurement $z_{snr}$, which is then incorporated into the state update via a Kalman filter or a similar estimation algorithm. The Jacobian matrix $\mathbf{H}_{snr,z,k}$ links the predicted state to the measurements, and the noise parameters $\mathbf{V}_{snr,z,k}$ and $\mathbf{R}_{snr,z,k}$ quantify the measurement uncertainty.

$$\mathbf{Y}_{snr,z,k} = z_{snr} = z_{k|k-1} + \varepsilon_{snr,k} \tag{19}$$

$$\mathbf{H}_{snr,z,k} = \frac{\partial \mathbf{Y}_{snr,z,k}}{\partial \hat{\mathbf{X}}_{z,k|k-1}} = \begin{bmatrix} \frac{\partial z_{snr}}{\partial z_{k|k-1}} & \frac{\partial z_{snr}}{\partial \dot{z}_{k|k-1}} \end{bmatrix} = \begin{bmatrix} 1 & 0 \end{bmatrix} \tag{20}$$

$$\mathbf{V}_{snr,z,k} = 1 \tag{21}$$

$$\mathbf{R}_{snr,z,k} = \sigma^2_{snr,z} \tag{22}$$

### 4.7 Heading Correction Using Magnetic Compass

A magnetic compass is used to measure the strength and direction of magnetic fields, typically to determine the orientation or heading of an object (e.g., a drone or vehicle) relative to the Earth's magnetic field. The sensor provides essential information for navigation systems, especially in combination with other sensors like gyroscopes and accelerometers.

In a drone or robot, a magnetic compass helps estimate the orientation (heading) of the device in 3D space. It measures the magnetic flux and calculates the azimuth (or yaw) angle, which indicates the direction of travel or the drone's orientation relative to the Earth's magnetic poles.

$$\mathbf{Y}_{mgn,\psi,k} = \psi_{mgn} = f_{mgn}(x_{mgn}, y_{mgn}) + \varepsilon_{mgn,k} = \psi_{k|k-1} + \varepsilon_{mgn,k} \tag{23}$$

$$\mathbf{H}_{mgn,\psi,k} = \begin{bmatrix} 1 & 0 \end{bmatrix} \tag{24}$$

$$\mathbf{V}_{mgn,\psi,k} = 1 \tag{25}$$

$$\mathbf{R}_{mgn,\psi,k} = \sigma^2_{mgn,\psi} \tag{26}$$

By incorporating the magnetic compass with the above equations 4.7, the sensor contributes to estimating the yaw angle in a drone. The magnetic compass measurement provides important information about the device's heading, and the system uses this data to update its state estimate while accounting for noise and uncertainty.

## 5 Conclusion & Improvement

### 5.1 Conclusion

In this project, we successfully implemented an autonomous UAV system that performs waypoint navigation relative to a moving ground robot (TurtleBot), using only onboard sensor data and ROS2 infrastructure. The overall architecture consisted of a modular design with three primary nodes: a behavior planner, a controller, and an estimator. Each component was carefully designed to interact with the others while maintaining independent functionality.

The **Behavior Node** adopted a finite state machine to sequence actions such as take-off, tracking, and landing based on dynamic input from the TurtleBot. The **Controller Node** used a Pure Pursuit algorithm to generate smooth velocity commands by identifying lookahead points, ensuring trajectory tracking even when the target was in motion. The **Estimator Node**, which forms the core of our localization system, fused multiple sensors including IMU and GPS using a simplified Extended Kalman Filter (EKF). The prediction step used IMU data to propagate the state, while the correction step adjusted the estimates with GPS data transformed from ECEF to the local ENU and world frames.

Overall, the system achieved stable and robust performance in simulation, with the drone accurately following the TurtleBot and responding effectively to state transitions. This validated both the estimation accuracy of our EKF and the smooth integration of behavior and control logic.

## 5.2 Improvement

While the project met its goals, several aspects could be enhanced in future iterations:

- **Sensor Noise Modeling:** The current EKF implementation assumes fixed process and measurement noise covariances. In reality, noise characteristics vary over time and across environments. 1 Incorporating adaptive noise tuning or learning-based estimation methods could improve robustness.

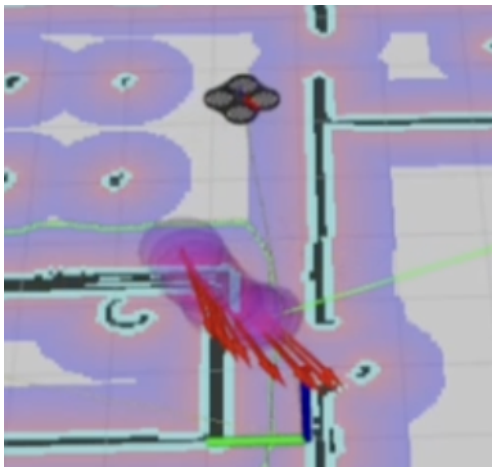

Figure 1: Sensor Noise

- **Yaw Estimation Accuracy:** The yaw update solely relies on gyroscope integration, which accumulates drift over time. Fusing additional sources such as magnetometer readings or visual odometry could mitigate drift and yield more accurate heading estimates.
- **Multi-Sensor Fusion:** Only IMU and GPS were used for state estimation. Future versions could incorporate barometer data for altitude correction and magnetometer for orientation reference, extending the EKF to a full 9-DOF system.
- **Failure Recovery and Fault Handling:** Currently, the behavior node assumes ideal conditions. Adding timeouts, fallback states, or anomaly detection mechanisms (e.g., GPS signal loss or outliers) would enhance system resilience.
- **Real-World Deployment Readiness:** Though the system was tested in simulation, the algorithms should be ported and validated on real hardware. This would require

handling communication delays, asynchronous sensor updates, and hardware-in-the-loop considerations.

- **Trajectory Prediction:** The current Pure Pursuit controller reacts to the TurtleBot's current position. Integrating trajectory prediction of the TurtleBot could improve drone path planning and reduce latency in reactive behavior. 2 []

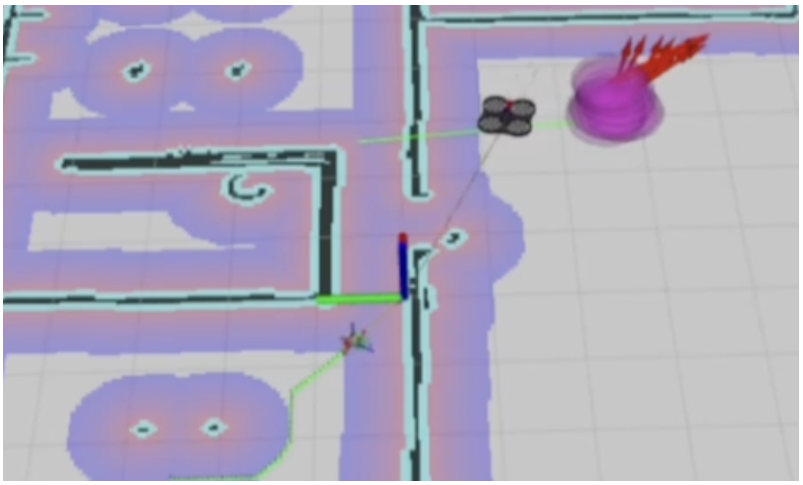

Figure 2: Trajectory Latency

- **Parameter Auto-Tuning:** Many parameters, such as lookahead distance, velocity limits, and FSM thresholds, were manually tuned. Implementing auto-tuning based on performance feedback could improve adaptability to different tasks and environments.

These improvements, if implemented, could significantly increase the applicability and reliability of the system in real-world autonomous robotic tasks.